

# Lab Experiment: 09

1st Assignment: Binary Tree Creation

Using Arrays:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX_SIZE 100

// Function to insert a value into the binary tree array
void insert(int tree[], int* size, int value) {
    if (*size >= MAX_SIZE) {
        printf("Binary tree is full, cannot insert more elements.\n");
        return;
    }
    // Insert the new value at the end of the current array
    tree[*size] = value;
    (*size)++;
    printf("Inserted %d into the binary tree.\n", value);
}

// Function to display the binary tree array representation
void displayTree(int tree[], int size) {
    if (size == 0) {
        printf("Binary tree is empty.\n");
        return;
    }

    printf("Binary tree in array representation:\n");
    for (int i = 0; i < size; i++) {
```

```

        printf("%d ", tree[i]);
    }
    printf("\n");
}

int main() {
    int tree[MAX_SIZE]; // Array to represent the binary tree
    int size = 0;      // Current number of nodes in the binary tree
    int choice, value;
    do {
        printf("\n--- Binary Tree Menu ---\n");
        printf("1. Insert Element\n");
        printf("2. Display Tree\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                insert(tree, &size, value);
                break;
            case 2:
                displayTree(tree, size);
                break;
            case 3:
                printf("Exiting...\n");
                break;
            default:

```

```
        printf("Invalid choice. Please try again.\n");
    }
} while (choice != 4);

return 0;
}
```

```
--- Binary Tree Menu ---
1. Insert Element
2. Display Tree
3. Exit
Enter your choice: 1
Enter value to insert: 11
Inserted 11 into the binary tree.

--- Binary Tree Menu ---
1. Insert Element
2. Display Tree
3. Exit
Enter your choice: 1
Enter value to insert: 21
Inserted 21 into the binary tree.

--- Binary Tree Menu ---
1. Insert Element
2. Display Tree
3. Exit
Enter your choice: 1
Enter value to insert: 31
Inserted 31 into the binary tree.

--- Binary Tree Menu ---
1. Insert Element
2. Display Tree
3. Exit
Enter your choice: 2
Binary tree in array representation:
11 21 31

--- Binary Tree Menu ---
1. Insert Element
2. Display Tree
3. Exit
Enter your choice: 3
Exiting...
```

Using Linked Lists:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct TreeNode {
```

```
    int data;
```

```
    struct TreeNode* left;
```

```
    struct TreeNode* right;
```

```
};
```

```
// Function to create a new node
```

```
struct TreeNode* createNode(int value) {
```

```
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
```

```
    if (newNode == NULL) {
```

```
        printf("Memory allocation failed\n");
```

```
        exit(1);
```

```
    }
```

```
    newNode->data = value;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to insert a new node in the binary tree
```

```
struct TreeNode* insertNode(struct TreeNode* root, int value) {
```

```
    if (root == NULL) {
```

```
        return createNode(value); // Create and return new node if root is null
```

```
    }
```

```
// Insert node in the left or right subtree
```

```
if (value < root->data) {
```

```
    root->left = insertNode(root->left, value);
```

```

    } else if (value > root->data) {
        root->right = insertNode(root->right, value);
    } else
        printf("Value already exists in the tree.\n");
    return root;
}

// display the tree
void inOrderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        inOrderTraversal(root->left);
        printf("%d ", root->data);
        inOrderTraversal(root->right);
    }
}

int main() {
    struct TreeNode* root = NULL;
    int choice, value;
    do {
        printf("\n--- Binary Tree Menu ---\n");
        printf("1. Insert Node\n");
        printf("2. Display Tree (In-order Traversal)\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                root = insertNode(root, value);

```

```

        break;
    case 2:
        printf("In-order traversal of the binary tree:\n");
        inOrderTraversal(root);
        printf("\n");
        break;
    case 3:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice. Please try again.\n");
    }
} while (choice != 4);
return 0;
}

```

```

--- Binary Tree Menu ---
1. Insert Node
2. Display Tree (In-order Traversal)
3. Exit
Enter your choice: 1
Enter value to insert: 5

--- Binary Tree Menu ---
1. Insert Node
2. Display Tree (In-order Traversal)
3. Exit
Enter your choice: 1
Enter value to insert: 10

--- Binary Tree Menu ---
1. Insert Node
2. Display Tree (In-order Traversal)
3. Exit
Enter your choice: 1
Enter value to insert: 15

--- Binary Tree Menu ---
1. Insert Node
2. Display Tree (In-order Traversal)
3. Exit
Enter your choice: 2
In-order traversal of the binary tree:
5 10 15

--- Binary Tree Menu ---
1. Insert Node
2. Display Tree (In-order Traversal)
3. Exit
Enter your choice: 3
Exiting...

```

## 2nd Assignment: Tree Traversal Methods

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct TreeNode {
```

```
    int data;
```

```
    struct TreeNode* left;
```

```
    struct TreeNode* right;
```

```
};
```

```
// Function to create a new node
```

```
struct TreeNode* createNode(int value) {
```

```
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
```

```
    newNode->data = value;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
// In-order traversal: Left, Root, Right
```

```
void inOrderTraversal(struct TreeNode* root) {
```

```
    if (root != NULL) {
```

```
        inOrderTraversal(root->left);
```

```
        printf("%d ", root->data);
```

```
        inOrderTraversal(root->right);
```

```
    }
```

```
}
```

```
// Pre-order traversal: Root, Left, Right
```

```
void preOrderTraversal(struct TreeNode* root) {
```

```
    if (root != NULL) {
```

```

        printf("%d ", root->data);
        preOrderTraversal(root->left);
        preOrderTraversal(root->right);
    }
}

// Post-order traversal: Left, Right, Root
void postOrderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        postOrderTraversal(root->left);
        postOrderTraversal(root->right);
        printf("%d ", root->data);
    }
}

// Level-order traversal (Breadth-first traversal) using a queue
void levelOrderTraversal(struct TreeNode* root) {
    if (root == NULL) return;

    // Define a simple queue using an array
    struct TreeNode* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        struct TreeNode* current = queue[front++];
        printf("%d ", current->data);
        if (current->left != NULL) queue[rear++] = current->left;
        if (current->right != NULL) queue[rear++] = current->right;
    }
}

```



```

// inserting a node (simple binary tree insertion)

struct TreeNode* insertNode(struct TreeNode* root, int value) {
    if (root == NULL)
        return createNode(value);
    if (value < root->data) {
        root->left = insertNode(root->left, value);
    } else if (value > root->data) {
        root->right = insertNode(root->right, value);
    } else
        printf("Value already exists in the tree.\n");
    return root;
}

int main() {
    struct TreeNode* root = NULL;
    int choice, value;
    do {
        printf("\n--- Binary Tree Menu ---\n");
        printf("1. Insert Node\n");
        printf("2. In-order Traversal\n");
        printf("3. Pre-order Traversal\n");
        printf("4. Post-order Traversal\n");
        printf("5. Level-order Traversal\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:

```

```
printf("Enter value to insert: ");  
scanf("%d", &value);  
root = insertNode(root, value);  
break;
```

case 2:

```
printf("In-order traversal: ");  
inOrderTraversal(root);  
printf("\n");  
break;
```

case 3:

```
printf("Pre-order traversal: ");  
preOrderTraversal(root);  
printf("\n");  
break;
```

case 4:

```
printf("Post-order traversal: ");  
postOrderTraversal(root);  
printf("\n");  
break;
```

case 5:

```
printf("Level-order traversal: ");  
levelOrderTraversal(root);  
printf("\n");  
break;
```

case 6:

```
printf("Exiting...\n");  
break;
```

default:

```
printf("Invalid choice. Please try again.\n");
```

```

    }
} while (choice != 6);

return 0;
}

```

```

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 1
Enter value to insert: 12

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 1
Enter value to insert: 3

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 1
Enter value to insert: 55

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 1
Enter value to insert: 7

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 2
In-order traversal: 3 7 12 55

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 3
Pre-order traversal: 12 3 7 55

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 4
Post-order traversal: 7 3 55 12

--- Binary Tree Menu ---
1. Insert Node
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Level-order Traversal
6. Exit
Enter your choice: 5
Level-order traversal: 12 3 55 7

```